COMPUTATIONAL PHYSICS

Y. D. Chong
Nanyang Technological University

LibreTexts™

Nanyang Technological University

Computational Physics

Y. D. Chong

This text is disseminated via the Open Education Resource (OER) LibreTexts Project (https://LibreTexts.org) and like the thousands of other texts available within this powerful platform, it is freely available for reading, printing, and "consuming."

The LibreTexts mission is to bring together students, faculty, and scholars in a collaborative effort to provide an accessible, and comprehensive platform that empowers our community to develop, curate, adapt, and adopt openly licensed resources and technologies; through these efforts we can reduce the financial burden born from traditional educational resource costs, ensuring education is more accessible for students and communities worldwide.

Most, but not all, pages in the library have licenses that may allow individuals to make changes, save, and print this book. Carefully consult the applicable license(s) before pursuing such effects. Instructors can adopt existing LibreTexts texts or Remix them to quickly build course-specific resources to meet the needs of their students. Unlike traditional textbooks, LibreTexts' web based origins allow powerful integration of advanced features and new technologies to support learning.

LibreTexts is the adaptable, user-friendly non-profit open education resource platform that educators trust for creating, customizing, and sharing accessible, interactive textbooks, adaptive homework, and ancillary materials. We collaborate with individuals and organizations to champion open education initiatives, support institutional publishing programs, drive curriculum development projects, and more.

The LibreTexts libraries are Powered by NICE CXone Expert and was supported by the Department of Education Open Textbook Pilot Project, the California Education Learning Lab, the UC Davis Office of the Provost, the UC Davis Library, the California State University Affordable Learning Solutions Program, and Merlot. This material is based upon work supported by the National Science Foundation under Grant No. 1246120, 1525057, and 1413739.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation nor the US Department of Education.

Have questions or comments? For information about adoptions or adaptions contact info@LibreTexts.org or visit our main website at https://LibreTexts.org.

This text was compiled on 06/09/2026

TABLE OF CONTENTS

Licensing

*A detailed breakdown of this resource's licensing can be found in **Back Matter/Detailed Licensing**.*

CHAPTER OVERVIEW

1: Scipy Tutorial

This is a tutorial for Scientific Python (Scipy), a scientific computing module for the Python programming language. There are a couple of other introductions to Scipy online, which are of excellent quality:

- Scipy Tutorial: the official tutorial.
- More Python Scientific Lecture Notes: a textbook which goes in-depth into using Scipy.

The present tutorial serves a slightly different purpose. It acts as a "walkthrough", guiding you through each step of writing a basic but complete Scipy program. You can use this as the basis for a more complete exploration of Scipy, possibly using the above online resources.

I will assume no pre-existing knowledge of the Python programming language. Programming language constructs are explained as they appear. But if you need more an even more basic tutorial on Python, feel free to consult any of the dozens available online.

1.1: Preliminaries

1.1.1 Installing Python and Scipy

If you don't have Scipy installed yet, there are plenty of installation options, detailed here.

- If you are using GNU/Linux, Python is probably already installed, so just install Scipy using your distribution's package manager (e.g. `apt-get install python3-scipy` for Debian or Ubuntu).
- If you are using Windows or Mac OS, the easiest installation method is to use the Anaconda distribution, which bundles Python with Scipy and other packages you might need. Pick the 64-bit Python 3.5 version.

From now on, I'll assume that you have installed Python 3, which is the newest version of the Python programming language. The old version, Python 2, also supports Scipy, but it brings along lots of little differences, too many and annoying to enumerate. All new (non-legacy) Python code ought to be written in Python 3.

1.1.2 Verify the Installation

If you are using GNU/Linux, open up a text terminal and type `python`. If you are using Windows, launch the program `Python 3.3 → IDLE (Python GUI)`. In each case, this will open up a text terminal with contents like this:

```
Python 3.3.3 (default, Nov 26 2013, 13:33:18)
[GCC 4.8.2] on linux
Type "copyright", "credits" or "license()" for more information.
>>>
```

The `>>>` part is a command prompt. Type the following:

```
>>> from scipy import *
```

After pressing Enter, there should be a brief pause, after which you get back to the prompt. (If you see a message like `ImportError: No module named 'scipy'`, then Scipy was not installed correctly.) Next, type

```
>>> import matplotlib.pyplot as plt
```

Again, there should be no error message. These two commands initialize the Scipy scientific computing module, and the Matplotlib plotting module, so that they are now available for use in Python. Note: in the future, you don't have to type these lines in by hand when starting up Python; we'll do all the necessary "importing" commands in our program source code.

Now let's do a simple plot of $y = \sin(x)$:

```
>>> x = linspace(0, 10, 100)
>>> y = sin(x)
>>> plt.plot(x,y)
>>> plt.show()
```

This should pop up a graph showing a sine function, in a window titled "Figure 1". Here's what these four lines of code did:

1. Create an array (a sequence of numbers), consisting of 100 numbers between 0 and 10, inclusive; then give this array the name `x`.
2. Create an array whose elements are the sines of the elements in `x`; i.e., a sequence of 100 numbers, the first of which is $\sin(0)$ and the last of which is $\sin(10)$. Then, give this array the name `y`.
3. Set up an $x - y$ plot, using the `x` array as the set of x coordinates, and the `y` array as the set of y coordinates.
4. Show the plot on-screen.

If you don't understand *why* the above lines do what they do, don't worry. Let's just keep going for now.

1.2: Getting Started

1.2.1: Problem Statement: Computing Electric Potentials

Let's now walk through the steps of writing a program to perform a simple task: computing and plotting the electric potential of a set of point charges located in a one-dimensional (1D) space.

Suppose we have a set of N point charges distributed in 1D. Let us denote the positions of the particles by $\{x_0, x_1, \cdots, x_{N-1}\}$, and their electric charges by $\{q_0, q_1, \cdots, q_{N-1}$. Notice that we have chosen to start counting from zero, so that x_0 is the first position and x_{N-1} is the last position. This practice is called "zero-based indexing"; more on it later.

Knowing $\{x_j\}$ and $\{q_j\}$, we can calculate $\phi(x)$ at any arbitrary point x, by using the formula

$$\phi(x) = \sum_{j=0}^{N-1} \frac{q_j}{4\pi\epsilon_0 |x - x_j|} \tag{1.2.1}$$

The factor of $4\pi\epsilon_0$ in the denominator is annoying to keep around, so we will adopt "computational units". This means that we'll rescale the potential, positions and/or the charges so that, in the new units of measurement, $4\pi\epsilon_0 = 1$. Then the formula for the potential simplifies to

$$\phi(x) = \sum_{j=0}^{N-1} \frac{q_j}{|x - x_j|} \tag{1.2.2}$$

Our goal now is to write a computer program which takes a set of positions and charges as its input, and plots the resulting electric potential.

1.2.2: Writing into a Python Source Code File

Before writing any Python code, we should create a file to put the code in. On GNU/Linux, fire up your preferred text editor and open an empty file, naming it `potentials.py` . On Windows, in the window that was opened up by the `IDLE (Python GUI)` program, click on the menu-bar item `File` → `New File` ; then type `Ctrl-s` (or click on `File` → `New File`) and save the empty file as `potentials.py` .

The file extension `.py` denotes it as a Python source code file. You should save the file in an appropriate directory.

Now let's do a very crude "first pass" at the program. Instead of handling an arbitrary number of particles, let's assume there's a *single* particle with some position x_0 and charge q_0. Then we'll plot its potential. Type the following into the file:

```
from scipy import *
import matplotlib.pyplot as plt

x0 = 1.5
q0 = 1.0

X   = linspace(-5, 5, 500)
phi = q0 / abs(X - x0)

plt.plot(X, phi)
plt.show()
```

Save the file. Now we will run the program. On GNU/Linux, open a text terminal and `cd` to the directory where you file is, then type `python -i potentials.py` . On Windows, while in file-editing window type `F5` (or click on `Run` → `Run Module`). In either case, you should see a figure pop up:

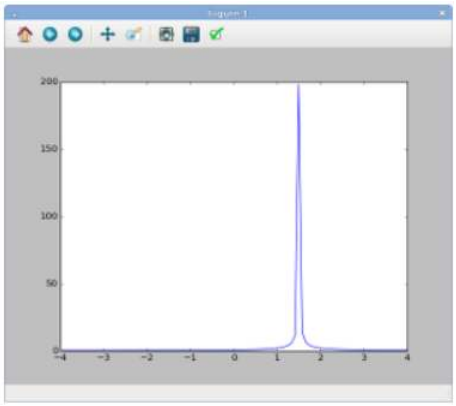

Figure 1.2.1

That's pretty much what we expect: the potential is peaked at $X = 1.5$, which is the position of the particle we specified in the program (via the variable named `x0`). The charge of the particle is given by the variable named `q0`, and we have assigned that the value 1.0. Hence, the potential is positive.

Now close the figure, and return to the Python command prompt. Note that Python is still running, even though your program has finished. From the command line, you can also examine the values of the variables which have been created by your program, simply by typing their names into the command prompt:

```
>>> x0
1.5
>>> phi
array([ 0.15384615    0.15432194    0.15480068    0.1552824  ....
          ....        0.28902404    0.28735963    0.28571429 ])
```

The value of `x0` is a number, 1.5, which was assigned to it when our program ran. The value of `phi` is more complicated: it is an *array*, which is a special data structure containing a sequence of numbers. From the command line, you can inspect the individual elements of this array. For example, to see the value of the array's first element, type this:

```
>>> phi[0]
0.153846153846
```

As we've mentioned, *index 0 refers to the first element of the array*. This so-called **zero-based indexing** is a common practice in computing. Similarly, index 1 refers to the second element of the array, index 2 refers to the third element, etc.

You can also look at the length of the array, by calling the function `len`. This function accepts an array input and returns its length, as an integer.

```
>>> len(phi)
500
```

You can exit the Python command line at any time by typing `Ctrl-d` or `exit()`.

1.3: Modularizing the Code

1.3.1: Designing a Potential Function

We could continue altering the above code in a straightforward way. For example, we could add more particles by adding variables `x1` , `x2` , `q1` , `q2` , and so forth, and altering our formula for computing `phi` . However, this is not very satisfactory: each time we want to consider a new collection of particle positions or charges, or change the number of particles, we would have to re-write the program's internal "logic"—i.e., the part that computes the potentials. In programming terminology, our program is insufficiently "modular". Ideally, we want to isolate the part of the program that computes the potential from the part that specifies the numerical inputs to the calculation, like the positions and charges.

To modularize the code, let's define a function that computes the potential of an *arbitrary* set of charged particles, sampled at an *arbitrary* set of positions. Such a function would need three sets of inputs:

- An array of particle positions $\vec{x} \equiv [x_0, \cdots, x_{N-1}]$. (Don't get confused, by the way: we are using these N numbers to refer to the positions of N particles in a 1D space, *not* the position of a single particle in an N-dimensional space.)
- An array of particle charges $\vec{q} \equiv [q_0, \cdots, q_{N-1}]$.
- An array of sampling points $\vec{X} \equiv [X_0, \cdots, X_{M-1}]$, which are the points where we want to know $\phi(X)$.

The number of particles, N, and the number of sampling points, M, should be arbitrary positive integers. Furthermore, N and M need not be equal.

The function we intend to write must compute the array

$$\begin{bmatrix} \phi(X_0) \\ \phi(X_1) \\ \vdots \\ \phi(X_{M-1}) \end{bmatrix} \tag{1.3.1}$$

which contains the value of the total electric potential at each of the sampling points. The total potential can be written as the sum of contributions from all particles. Let us define $\phi_j(x)$ as the potential produced by particle j:

$$\phi_{j(x)} \equiv \frac{q_j}{|x - x_j|} \tag{1.3.2}$$

Then the total potential is

$$\begin{bmatrix} \phi(X_0) \\ \phi(X_1) \\ \vdots \\ \phi(X_{M-1}) \end{bmatrix} = \begin{bmatrix} \phi_0(X_0) \\ \phi_0(X_1) \\ \vdots \\ \phi_0(X_{M-1}) \end{bmatrix} + \begin{bmatrix} \phi_1(X_0) \\ \phi_1(X_1) \\ \vdots \\ \phi_1(X_{M-1}) \end{bmatrix} + \cdots + \begin{bmatrix} \phi_{N-1}(X_0) \\ \phi_{N-1}(X_1) \\ \vdots \\ \phi_{N-1}(X_{M-1}) \end{bmatrix}. \tag{1.3.3}$$

1.3.2 Writing the Program

Let's code this up. Return to the file `potentials.py` , and **delete the entire contents of the file**. Then replace it with the following:

```
from scipy import *
import matplotlib.pyplot as plt

## Return the potential at measurement points X, due to particles
## at positions xc and charges qc.  xc, qc, and X must be 1D arrays,
## with xc and qc of equal length.  The return value is an array
## of the same length as X, containing the potentials at each X point.
def potential(xc, qc, X):
    M = len(X)
    N = len(xc)
    phi = zeros(M)
```

```
    for j in range(N):
        phi += qc[j] / abs(X - xc[j])
    return phi

charges_x  = array([0.2, -0.2])
charges_q  = array([1.5, -0.1])
xplot = linspace(-3, 3, 500)

phi = potential(charges_x, charges_q, xplot)

plt.plot(xplot, phi)
pmin, pmax = -50, 50
plt.ylim(pmin, pmax)
plt.show()
```

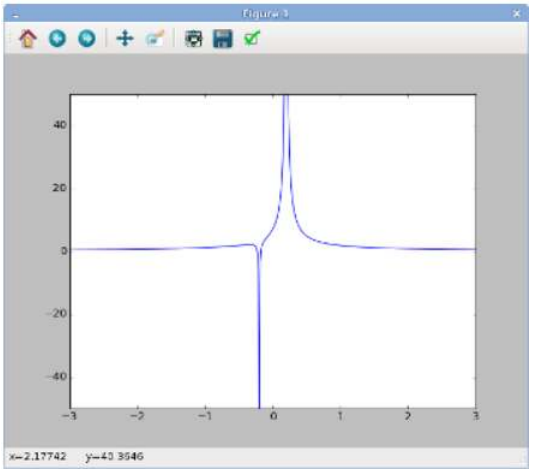

Figure 1.3.1

When typing or pasting the above into your file, be sure to preserve the **indentation** (i.e., the number of spaces at the beginning of each line). Indentation is important in Python; as we'll see, it's used to determine program structure. Now save and run the program again:

- In the Windows GUI, type `F5` in the editing window showing `potentials.py` .
- On GNU/Linux, type `python -i potentials.py` from the command line.
- Alternatively, from the Python command line, type `import potentials` , which will load and run your `potentials.py` file.

You should now see a figure like the one on the right, showing the electric potential produced by two particles, one at position $x_0 = 0.2$ with charge $q_0 = 1.5$ and the other at position $x_1 = -0.2$ with charge $q_1 = -0.1$.

There are less than 20 lines of actual code in the above program, but they do quite a lot of things. Let's go through them in turn:

Module Imports

The first two lines import the Scipy and Matplotlib modules, for use in our program. We have not yet explained how importing works, so let's do that now.

```
from scipy import *
import matplotlib.pyplot as plt
```

Each Python module, including Scipy and Matplotlib, defines a variety of functions and variables. If you use multiple modules, you might have a situation where, say, two different modules each define a function with the same name, but doing entirely different things. That would be Very Bad. To help avoid this, Python implements a concept called a **namespace**. Suppose you import a module (say Scipy) like this:

```
import scipy
```

One of the functions defined by Scipy is `linspace`, which we have already seen. This function was defined by the `scipy` module, and lies inside the `scipy` namespace. As a result, when you import the Scipy module using the `import scipy` line, you have to call the `linspace` function like this:

```
x = scipy.linspace(-3, 3, 500)
```

The `scipy.` in front says that you're referring to the `linspace` function that was defined in the `scipy` namespace. (Note: the online documentation for `linspace` refers to it as `numpy.linspace`, but the exact same function is also present in the `scipy` namespace. In fact, all `numpy.*` functions are replicated in the `scipy` namespace. So unless stated otherwise, we only have to import `scipy`.)

We will be using a lot of functions that are defined in the `scipy` namespace. Since it would be annoying to have to keep typing `scipy.` all over the place, we opt to use a slightly different import statement:

```
from scipy import *
```

This imports all the functions and variables in the `scipy` namespace directly into your program's namespace. Therefore, you can just call `linspace`, without the `scipy.` prefix. Obviously, you don't want to do this for every module you use, otherwise you'll end up with the name-clashing problem we alluded to earlier! The only module we'll use this shortcut with is `scipy`.

Another way to avoid having to type long prefixes is shown by this line:

```
import matplotlib.pyplot as plt
```

This imports the `matplotlib.pyplot` module (i.e., the `pyplot` module which is nested inside the `matplotlib` module). That's where `plot`, `show`, and other plotting functions are defined. The `as plt` in the above line says that we will refer to the `matplotlib.pyplot` namespace as the short form `plt` instead. Hence, instead of calling the `plot` function like this:

```
matplotlib.pyplot.plot(x, y)
```

we will call it like this:

```
plt.plot(x, y)
```

Comments

Let's return to the program we were looking at earlier. The next few lines, beginning with `#`, are "comments". Python ignores the `#` character and everything that follows it, up to the end of the line. Comments are very important, even in simple programs like this.

When you write your own programs, please remember to include comments. You don't need a comment for every line of code—that would be excessive—but at a minimum, each function should have a comment explaining what it does, and what the inputs and return values are.

Function Definition

Now we get to the *function definition* for the function named `potential`, which is the function that computes the potential:

```
def potential(xc, qc, X):
    M = len(X)
    N = len(xc)
    phi = zeros(M)
    for j in range(N):
        phi += qc[j] / abs(X - xc[j])
    return phi
```

The first line, beginning with `def`, is a **function header**. This function header states that the function is named `potential`, and that it has three inputs. In computing terminology, the inputs that a function accepts are called **parameters**. Here, the parameters are named `xc`, `qc` and `X`. As explained by the comments, we intend to use these for the positions of the particles, the charges of the particles, and the positions at which to measure the potential, respectively.

The function definition consists of the function header, together the rest of the *indented* lines below it. The function definition terminates once we get to a line which is at the same indentation level as the function header. (That terminating line is considered a separate line of code, which is not part of the function definition).

By convention, you should use 4 spaces per indentation level.

The indented lines below the function header are called the **function body**. This is the code that is run each time the function is called. In this case, the function body consists of six lines of code, which are intended to compute the total electric potential, according to the procedure that we have outlined in the preceding section:

- The first two lines define two helpful variables, `M` and `N`. Their values are set to the lengths of the `X` and `xc` arrays, respectively.
- The next line calls the `zeros` function. The input to `zeros` is `M`, the length of the `X` array (i.e., our function's third parameter). Therefore, `zeros` returns an array, of the same same length as `X`, with every element set to 0.0. For now, this represents the electric potential in the absence of any charges. We give this array the name `phi`.
- The function then iterates over each of the particles and add up its contribution to the potential, using a construct known as a `for` loop. The code `for j in range(N):` is the loop's "header line", and the next line, indented 4 spaces deeper than the header line, is the "body" of the loop.

  The header line states that we should run the loop body several times, with the variable `j` set to different values during each run. The values of `j` to loop over are given by `range(N)`. This is a function call to the range function, with `N` (the number of electric charges) as the input. The `range(N)` function call returns a sequence specifying `N` successive integers, from 0 to `N-1`, inclusive. (Note that the last value in the sequence is `N-1`, not `N`. Because we start from 0, this means that there is a total of `N` integers in the sequence. Also, calling `range(N)` is the same as calling `range(0,N)`.)
- For each `j`, we compute `qc[j] / abs(X - xc[j])`. This is an array whose elements are the values of the electric potential at the set of positions `X`, arising from the individual particle$j$. In mathematical terms, we are calculating

  $$\phi_j(X) \equiv \frac{q_j}{|X - x_j|} \tag{1.3.4}$$

  using the array of positions `X`. We then add this array to `phi`. Once this is done for all `j`, the array `phi` will contain the desired total potential,

  $$\phi(X) = \sum_{j=0}^{N-1} \phi_j(X). \tag{1.3.5}$$
- Finally, we call `return` to specify the function's output, or **return value**. This is the array `phi`.

Top-Level Code: Numerical Constants

After the function definition comes the code to use the function:

```
charges_x  = array([0.2, -0.2])
charges_q  = array([1.5, -0.1])
xplot = linspace(-3, 3, 500)
```

Like the import statements at the beginning of the program, these lines of code lie at **top level**, i.e., they are not indented. The function header which defines the `potential` function is also at top level. Running a Python program consists of running its top level code, in sequence.

The above lines define variables to store some numerical constants. In the first two lines, `charges_x` and `charges_q` variables store the numerical values of the positions and charges we are interested in. These are initialized using the `array` function. You may be wondering why the `array` function call has square brackets nested in commas. We'll explain later, in part 2 of the tutorial.

On the third line, the `linspace` function call returns an array, whose contents are initialized to the 500 numbers between -3 and 3 (inclusive).

Next, we call the `potential` function, passing `charges_x` , `charges_q` and `xplot` as the inputs:

```
phi = potential(charges_x, charges_q, xplot)
```

These inputs provide the values of the function definition's parameters `xc` , `qc` , and `X` respectively. The return value of the function call is an array containing the total potential, evaluated at each of the positions specified in `xplot` . This return value is stored as the variable named `phi` .

Plotting

Finally, we create the plot:

```
plt.plot(xplot, phi)
pmin, pmax = -50, 50
plt.ylim(pmin, pmax)
plt.show()
```

We have already seen how the `plot` and `show` functions work. Here, prior to calling `plt.show` , we have added two extra lines to make the potential curve is more legible, by adjust the plot's y-axis bounds. The ylim function accepts two parameters, the lower and upper bounds of the y-axis. In this case, we set the bounds to -50 and 50 respectively. There is an xlim function to do the same for the x-axis.

Notice that in the line `pmin, pmax = -50, 50` , we set two variables (`pmin` and `pmax`) on the same line. This is a little "syntactic sugar" to make the code a little easier to read. It's equivalent to having two separate lines, like this:

```
pmin = -50
pmax = 50
```

We'll explain how this construct works in the next part of the tutorial.

CHAPTER OVERVIEW

2: Scipy Tutorial (Part 2)

This is part 2 of the Scientific Python tutorial.

2.1: Sequential Data Structures

In the previous part of the tutorial, we worked through a simple example of a Scipy program which calculates the electric potential produced by a collection of charges in 1D. At the time, we did not explain much about the data structures that we were using to store numerical information (such as the values of the electric potential at various points). Let's do that now.

There are three common data structures that will be used in a Scipy program for scientific computing: arrays, lists, and tuples. These structures store linear sequences of Python objects, similar to the concept of "vectors" in physics and mathematics. However, these three types of data structures all have slightly different properties, which you should be aware of.

2.1.1 Arrays

An array is a data structure that contains a sequence of numbers. Let's do a quick recap. From a fresh Python command prompt, type the following:

```
>>> from scipy import *
>>> x = linspace(-0.5, 0.5, 9)
>>> x
array([-0.5  , -0.375, -0.25 , -0.125,  0.   ,  0.125,  0.25 ,  0.375,  0.5  ])
```

The first line, as usual, is used to import the `scipy` module. The second line creates an array named `x` by calling linspace, which is a function defined by `scipy`. With the given inputs, the function returns an array of 9 numbers between -0.5 and 0.5, inclusive. The third line shows the resulting value of `x`.

The array data structure is provided specifically by the Scipy scientific computing module. Arrays can only contain numbers (furthermore, each individual array can only contain numbers of one type, e.g. integers or complex numbers; we'll discuss this in the next article). Arrays also support special facilities for doing numerical linear algebra. They are commonly created using one of these functions from the `scipy` module:

- linspace, which creates an array of evenly-spaced values between two endpoints.
- arange, which creates an array of integers in a specified range.
- zeros, which creates an array whose elements are all 0.0.
- ones, which creates an array whose elements are all 1.0.
- empty, which creates an array whose elements are uninitialized (this is usually used when you want to set the elements later).

Of these, we've previously seen examples of the `linspace` and `zeros` functions being used. As another example, to create an array of 500 elements all containing the number -1.2, you can use the `ones` function and a multiplication operation:

```
x = -1.2 * ones(500)
```

An alternative method, which is *slightly* faster, is to generate the array using `empty` and then use the `fill` method to populate it:

```
x = empty(500); x.fill(-1.2)
```

One of the most important things to know about an array is that its size is fixed at the moment of its creation. When creating an array, you need to specify exactly how many numbers you want to store. If you ever need to revise this size, you must create a *new* array, and transfer the contents over from the old array. (For very big arrays, this might be a slow operation, because it involves copying a lot of numbers between different parts of the computer memory.)

You can pass arrays as inputs to functions in the usual way (e.g., by supplying its name as the argument to a function call). We have already encountered the `len` function, which takes an array input and returns the array's length (an integer). We have also encountered the `abs` function, which accepts an array input and returns a new array containing the corresponding absolute values. Similar to `abs`, many mathematical functions and operations accept arrays as inputs; usually, this has the effect of applying the function or operation to each element of the input array, and returning the result as another array. The returned array

has the same size as the input array. For example, the `sin` function with an array input `x` returns another array whose elements are the sines of the elements of `x` :

```
>>> y = sin(x)
>>> y
array([-0.47942554, -0.36627253, -0.24740396, -0.12467473,  0.        ,
        0.12467473,  0.24740396,  0.36627253,  0.47942554])
```

You can access individual elements of an array with the notation `a[j]` , where `a` is the variable name and `j` is an integer index (where the first element has index 0, the second element has index 1, etc.). For example, the following code sets the first element of the `y` array to the value of its second element:

```
>>> y[0] = y[1]
>>> y
array([-0.36627253, -0.36627253, -0.24740396, -0.12467473,  0.        ,
        0.12467473,  0.24740396,  0.36627253,  0.47942554])
```

Negative indices count backward from the end of the array. For example:

```
>>> y[-1]
0.47942553860420301
```

Instead of setting or retrieving individual values of an array, you can also set or retrieve a *sequence* of values. This is referred to as **slicing**, and is described in detail in the Scipy documentation. The basic idea can be demonstrated with a few examples:

```
>>> x[0:3] = 2.0
>>> x
array([ 2.   ,  2.   ,  2.   , -0.125,  0.   ,  0.125,  0.25 ,  0.375,  0.5  ])
```

The above code accesses the elements in array `x` , starting from index 0 up to but not including 3 (i.e. indices 0, 1, and 2), and assigns them the value of `2.0` . This changes the contents of the array `x` .

```
>>> z = x[0:5:2]
>>> z
array([ 2.,  2.,  0.])
```

The above code retrieves a subset of the elements in array `x` , starting from index 0 up to but not including 5, and stepping by 2 (i.e., the indices 0, 2, and 4), and then groups those elements into an array named `z` . Thereafter, `z` can be treated as an array.

Finally, arrays can also be multidimensional. If we think of an ordinary (1D) array as a vector, then a 2D array is equivalent to a matrix, and higher-dimensional arrays are like tensors. We will see practical examples of higher-dimensional arrays later. For now, here is a simple example:

```
>>> y = zeros((4,2))     # Create a 2D array of size 4x2
>>> y[2,0] = 1.0
>>> y[0,1] = 2.0
>>> y
array([[ 0.,  2.],
       [ 0.,  0.],
```

```
       [ 1.,  0.],
       [ 0.,  0.]])
```

2.1.2: Lists

There is another type of data structure called a **list**. Unlike arrays, lists are built into the Python programming language itself, and are not specific to the Scipy module. Lists are general-purpose data structures which are *not* optimized for scientific computing (for example, we will need to use arrays, not lists, when we want to do linear algebra).

The most convenient thing about Python lists is that you can specify them explicitly, using `[...]` notation. For example, the following code creates a list named `u`, containing the integers 1, 1, 2, 3, and 5:

```
>>> u = [1, 1, 2, 3, 5]
>>> u
[1, 1, 2, 3, 5]
```

This way of creating lists is also useful for creating Scipy arrays. The array function accepts a list as an input, and returns an array containing the same elements as the input list. For example, to create an array containing the numbers 0.2, 0.1, and 0.0:

```
>>> x = array([0.2, 0.1, 0.0])
>>> x
array([ 0.2,  0.1,  0. ])
```

In the first line, the square brackets create a list object containing the numbers 0.2, 0.1, and 0.0, then passes that list directly as the input to the `array` function. The above code is therefore equivalent to the following:

```
>>> inputlist = [0.2, 0.1, 0.0]
>>> inputlist
[0.2, 0.1, 0.0]
>>> x = array(inputlist)
>>> x
array([ 0.2,  0.1,  0. ])
```

Usually, we will do number crunching using arrays rather than lists. However, sometimes it is useful to work directly with lists. One convenient thing about lists is that they can contain arbitrary Python objects, of any data type; by contrast, arrays are allowed only to contain numerical data.

For example, a Python list can store character strings:

```
>>> u = [1, 2, 'abracadabra', 3]
>>> u
[1, 2, 'abracadabra', 3]
```

And you can set or retrieve individual elements of a Python list in the same way as an array:

```
>>> u[1] = 0
>>> u
[1, 0, 'abracadabra', 3]
```

Another great advantage of lists is that, unlike arrays, you can dynamically increase or decrease the size of a list:

```
>>> u.append(99)                    # Add 99 to the end of the list u
>>> u.insert(0, -99)                # Insert -99 at the front (index 0) of the list u
>>> u
[-99, 1, 0, 'abracadabra', 3, 99]
>>> z = u.pop(3)                    # Remove element 3 from list u, and name it z
>>> u
[-99, 1, 0, 3, 99]
>>> z
'abracadabra'
```

Aside: About Methods

In the above example, `append`, `insert`, and `pop` are called **methods**. You can think of a method as a special kind of function which is "attached" to an object and relies upon it in some way. Just like a function, a method can accept inputs, and give a return value. For example, when `u` is a list, the code

```
z = u.pop(3)
```

means to take the list `u`, find element index 3 (specified by the input to the method), remove it from the list, and return the removed list element. In this case, the returned element is named `z`. See here for a summary of Python list methods. We'll see more examples of methods as we go along.

2.1.3 Tuples

Apart from lists, Python provides another kind of data structure called a **tuple**. Whereas lists can be constructed using square bracket `[...]` notation, tuples can be constructed using parenthetical `(...)` notation:

```
>>> v = (1, 2, 'abracadabra', 3)
>>> v
(1, 2, 'abracadabra', 3)
```

Like lists, tuples can contain any kind of data type. But whereas the size of a list can be changed (using methods like `append`, `insert`, and `pop`, as described in the previous subsection), the size of a tuple is fixed once it's created, just like an array.

Tuples are mainly used as a convenient way to "group" or "ungroup" named variables. Suppose we want to split the contents of `v` into four separate named variables. We could do it like this:

```
>>> dog, cat, apple, banana = v
>>> dog
1
>>> cat
2
>>> apple
'abracadabra'
>>> banana
3
```

On the left-hand side of the `=` sign, we're actually specifying a tuple of four variables, named `dog`, `cat`, `apple`, and `banana`. In cases like this, it is OK to omit the parentheses; when Python sees a group of variable names separated by commas, it automatically treats that group as a tuple. Thus, the above line is equivalent to

```
>>> (dog, cat, apple, banana) = v
```

We saw a similar example in the previous part of the tutorial, where there was a line of code like this:

```
pmin, pmax = -50, 50
```

This assigns the value -50 to the variable named `pmin`, and 50 to the variable named `pmax`. We'll see more examples of tuple usage as we go along.
